

**"THE GUARDIAN": A MODULAR AI FOR
PROACTIVE HUMAN-LAYER CYBERSECURITY ON
SOCIAL MEDIA.**

Project ID: 25-26J-041

Project Proposal Report

Fernando M R I (IT22366054)

Supervisor – Kavinga Yapa Abeywardena

Co-supervisor – Deemantha Siriwardana

B. Sc. (Hons) Degree in Information Technology (Cyber Security)

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

**"THE GUARDIAN": A MODULAR AI FOR
PROACTIVE HUMAN-LAYER CYBERSECURITY ON
SOCIAL MEDIA.**

Project ID: 25-26J-041

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology (Cyber Security)

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

DECLARATION

I declare that this proposal is my own work, and it does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or institute of higher learning. To the best of my knowledge, it does not contain any material previously published or written by another person, except where such reference is made in the text.

Name: **Fernando M R I**

Student ID: **IT22366054**

Signature: _____

Date: 29/8/2025

The above candidate is carrying out this research for the undergraduate dissertation under my supervision.

Supervisor:

Signature of Supervisor: _____

Date: 29/8/2025

Abstract

The rapid growth of social media has increased human-layer cybersecurity risks, turning platforms into channels for hate speech, misinformation, scams, and privacy issues. This research proposes “*The Guardian*” — a modular AI-powered browser extension and backend API system that gives users real-time, non-intrusive analysis of text without censorship or invading privacy. The main problem is to build a system that uses advanced Natural Language Processing (NLP) to detect and explain subtle human-layer threats, focusing on hate speech, including hidden and context-based toxicity. The approach involves fine-tuning Transformer models on public datasets of labeled social media posts, supported by custom samples for harder cases. The system uses a FastAPI backend to host detection modules and a JavaScript-based browser extension to show color-coded icons with short explanations. The expected results are high accuracy in detecting hate speech, fewer false positives on normal critiques, and better user trust through clear, explainable alerts. The key contribution is a user-focused, privacy-safe tool that strengthens personal cybersecurity by guiding users against harmful language, with the hate speech module improving detection of subtle toxicity while ensuring fairness and reducing bias.

Keywords: Cybersecurity, Natural Language Processing, Human-layer cybersecurity, Hate speech detection.

TABLE OF CONTENTS

DECLARATION	i
Abstract.....	ii
List of Tables.....	v
1. INTRODUCTION	1
2. BACKGROUND & LITERATURE SURVEY	2
2.1 Human-centric threats on social media.....	2
2.2 Hate speech detection & classification	2
2.3 Underlying Technologies and Methodological Approach	3
2.4 The Hate Speech Detection Gap	3
3. RESEARCH GAP.....	4
4. RESEARCH PROBLEM.....	5
5. OBJECTIVES	6
5.1 Main Objective	6
5.2 Sub-Objectives.....	6
6. METHODOLOGY	8
6.1 System Architecture and Data Flow	8
6.2 User Flow.....	9
6.3 Data Acquisition and Preprocessing	11
6.4 Software Solution	11
7. PROJECT REQUIREMENTS.....	16
7.1 User Requirements.....	16
7.2 Functional Requirements (FR).....	17
7.3 Non-Functional Requirements (NFR)	18
7.4 Test Cases.....	18
8. BUDGET AND BUDGET JUSTIFICATION	20

REFERENCES	21
------------------	----

List of Tables

Table 7.2: Functional Requirements (FR)	15
Table 7.3: Non-Functional Requirements (NFR)	16
Table 7.4: Test Cases	16

1. INTRODUCTION

In the contemporary digital era, social media platforms have revolutionized connectivity but simultaneously escalated human-layer cybersecurity threats by exploiting psychological vulnerabilities, trust, and cognitive biases [1]. These threats manifest as coordinated misinformation undermining societal trust, sophisticated scams causing financial harm, inadvertent disclosures of Personally Identifiable Information (PII) leading to identity risks, and hate speech fueling harassment, violence, and division [1, 5, 15]. Conventional defenses, such as reactive platform moderation and traditional antivirus tools, are insufficient, often responding post-harm and failing to address the linguistic nuances of these threats [8, 11].

This research introduces "The Guardian," a proactive, modular AI-powered system comprising a lightweight browser extension frontend and a Python-FastAPI backend. The extension scrapes social media text in real-time, sending it to four specialized Natural Language Processing (NLP) modules for analysis: hate speech, misinformation, phishing/scams, and PII/OpSec. Results are returned as structured JSON responses, enabling non-intrusive overlays like color-coded icons with explanatory tooltips (e.g., "This post may contain implicit hate speech language").

Central to this proposal is the hate speech detection module, which targets toxic language, including subtle, context-dependent [3, 9]. By fine-tuning Transformer models on datasets for rationale-based explanations and for distinguishing hate speech from offensive language, the module enhances detection of slang and implicit biases while minimizing false positives on legitimate discourse [1, 2]. Additionally, incorporating social context and user data, as explored in recent studies, improves robustness in identifying nuanced toxicity across diverse social media platforms [6]. Bias mitigation, a critical aspect, ensures fairness by addressing unintended demographic biases in model predictions, aligning with ethical AI principles [14]. This complies with cybersecurity goals of threat analysis and mitigation, as hate speech and online harassment are potent vectors exacerbating cyberbullying's psychological toll, and automating their identification fosters safer online environments [1, 9].

The project leverages domain expertise in cybersecurity (Information Assurance, Social Engineering, Human-Computer Interaction), with proficiency in Python, Natural Language Process, AI tools, and Software implementations.

2. BACKGROUND & LITERATURE SURVEY

The evolution of social media has transformed communication, but it has also amplified human-centric threats through linguistic manipulation. This section surveys key literature on these threats, focusing on hate speech detection, core technologies, and gaps in current approaches.

2.1 Human-centric threats on social media

Social media platforms have become breeding grounds for human-centric threats, where psychological vulnerabilities are exploited through misinformation, scams, and hate speech [15]. Ghafir et al. (2018) highlight how trust-based social engineering undermines critical infrastructure, paralleling phishing and scam tactics on platforms [2]. Al-Rimy et al. (2018) provide a taxonomy of ransomware threats emphasizing human factors, which extend to broader online harms like privacy breaches and identity risks [1, 3]. These threats often intersect with hate speech, as coordinated campaigns erode societal trust and incite division [11]. Recent reviews underscore the proliferation of hate speech on platforms like Twitter, driven by anonymity and rapid dissemination [3, 12]. Oussalah (2023) notes that multilingual and context-dependent hate amplifies these risks, while Poletto et al. (2021) emphasizes the need for diverse corpora to address cultural nuances [8].

2.2 Hate speech detection & classification

Hate speech detection has advanced through supervised learning, focusing on explicit and implicit forms. Davidson et al. (2017) pioneered multi-class classification distinguishing hate from offensive language using Twitter datasets, achieving strong baselines but struggling with nuances [1]. Mathew et al. (2021) introduced HateXplain, a benchmark with explainable annotations for toxic spans, enabling interpretable detection [2]. Rawat et al. (2024) review techniques like Transformers for social media hate, highlighting trends in handling coded language [3]. Fortuna et al. (2021) evaluate model generalization across datasets, revealing poor transfer for implicit hate [4]. Mutanga et al. (2020) apply Transformers to Twitter hate, improving on RNNs for sequence modeling [5]. Nagar et al. (2023) incorporate social context

and user data for robust detection [6]. Huang et al. (2023) explored for explaining implicit hate, showing limitations in annotation quality [7]. Poletto et al. (2021) systematically reviews benchmark corpora, noting English dominance [8]. Wiegand et al. (2021) analyze implicit abuse subtypes like stereotypes, explaining detection challenges [9]. Hartvigsen et al. (2022) generate ToxiGen for adversarial implicit hate [10]. Oussalah (2023) surveys NLP pipelines for automatic detection [11]. Aluru et al. (2024) focus on Transformers' superiority in multilingual hate [12]. Das et al. (2023) extend to multimodal hate videos [13]. Nozza et al. (2019) address unintended bias in misogyny detection [14]. Arango et al. (2020) critique validation methods leading to overfitting [15]. Bohra et al. (2018) provide Hindi-English code-mixed data [16]. Zampieri et al. (2019) predict offense types and targets [17]. Vidgen and Derczynski (2020) discuss data quality issues [18]. Sheth et al. (2023) use LLMs for explainable detection [19]. Mayda et al. (2021) curate Turkish hate datasets [20].

2.3 Underlying Technologies and Methodological Approach

Core technologies in hate speech detection leverage NLP advancements, particularly Transformers and datasets. BERT variants like DistilBERT enable efficient fine-tuning for classification [5, 12]. Multi-task learning incorporates rationales for explainability [2, 19]. Graph-based methods integrate social context [6]. Datasets like Davidson's Twitter corpus [1] and HateXplain [2] provide benchmarks, while ToxiGen addresses implicit hate through machine generation [10]. Multilingual corpora, such as Hindi-English code-mixed [16] and Turkish [20], support low-resource detection [12]. Validation emphasizes cross-dataset generalization [4, 15], with bias mitigation via adversarial training [14, 18].

2.4 The Hate Speech Detection Gap

Gaps in hate speech detection include poor handling of implicit forms [9, 10], biases toward explicit keywords [1, 14], and limited generalization across datasets [4, 15]. Implicit abuse, like stereotypes, evades lexical methods [3, 9]. Multilingual and code-mixed data remain underrepresented [16, 20]. Multimodal extensions are nascent [13]. Data quality issues, such as biases and sparsity, hinder robustness [18].

3. RESEARCH GAP

Existing solutions for hate speech detection exhibit critical shortcomings, particularly in addressing subtle, context-dependent toxicity. This section identifies these gaps and justifies the need for proactive, user-centric systems like The Guardian.

- **What is missing in existing solutions:** Current models excel in explicit hate but falter on implicit forms [3, 9, 10]. Generalization across datasets is poor due to biases and overfitting [4, 15, 18]. Multilingual and code-mixed detection lags, with limited corpora for languages like Hindi-English [16] and Turkish [20]. Bias in misogyny and demographic targeting persists [14]. Multimodal approaches are underdeveloped [13]. Annotation inconsistencies and data quality issues exacerbate these problems [7, 8, 17].
- **Why ‘Guardian’, proactive guidance is needed:** Reactive moderation fails to prevent harm, necessitating proactive, non-intrusive tools [3, 11]. The Guardian's modular NLP architecture provides real-time, context-aware alerts without censorship, addressing gaps in user empowerment and privacy [12]. Its hate speech module targets subtle toxicity, improving accuracy and fairness beyond keyword-based methods [1, 2].

4. RESEARCH PROBLEM

The primary research problem is to explore the effective training and deployment of state-of-the-art NLP models within a modular architecture for real-time, context-aware threat analysis on social media, enabling non-intrusive user guidance to bolster personal cybersecurity without privacy or freedom compromises.

A focused sub-problem addresses hate speech detection: how to develop and evaluate an AI module that transcends basic keyword methods to identify implicit, context-dependent toxicity (e.g., "dog whistles" and slangs with using special characters), while enhancing accuracy, reducing false positives on legitimate critiques, and ensuring fairness through bias analysis. Tasks encompass researching hate speech linguistics, collecting/cleaning datasets (e.g., Davidson et al., 2017; HateXplain by Mathew et al., 2020; Rawat et al., 2024), fine-tuning Transformers, and evaluating biases.

This complies with cybersecurity via threat analysis and mitigation: hate speech as a vector for harassment and cyberbullying inflicts psychological damage, and automated detection fosters safer environments. Novelty resides in context-sensitive detection, integrating explainability (Mathew et al., 2020) and user indices (Krupalija et al., 2022) for equitable, low-false-positive outcomes.

5. OBJECTIVES

The primary goal of this research is to design, implement, and evaluate "The Guardian," a modular AI-powered system that proactively assists users in identifying and understanding human-layer cybersecurity threats on social media in a non-intrusive manner, enhancing their personal cybersecurity posture without compromising privacy or autonomy. The system leverages state-of-the-art Natural Language Processing (NLP) models within a browser extension and FastAPI backend to deliver real-time, context-aware analysis of text content, addressing threats including hate speech, misinformation, phishing/scams, and PII/OpSec risks. The hate speech detection component is central, focusing on detecting explicit and subtle toxicity, to mitigate online harassment and cyberbullying [1, 3, 9].

5.1 Main Objective

To develop and evaluate "The Guardian," a modular AI system that provides real-time, non-intrusive, context-aware guidance to social media users, enabling them to identify and mitigate human-layer cybersecurity threats, including hate speech, misinformation, phishing/scams, and PII/OpSec risks, while preserving user autonomy and privacy.

5.2 Sub-Objectives

Develop and Evaluate a Specialized Hate Speech Detection Module: To create and assess an AI module within "The Guardian" that detects explicit and implicit hate speech, including context-dependent forms, while improving accuracy and reducing false positives on legitimate critiques. This involves researching hate speech linguistics, collecting, and augmenting datasets (e.g., Davidson [1], HateXplain [2]), fine-tuning Transformer-based models (e.g., BERT, DistilBERT), and analyzing model biases to ensure fairness across diverse demographics and contexts [3, 9, 14].

Design a Modular AI Architecture: To architect a scalable system integrating four specialized NLP modules (hate speech, misinformation, phishing/scams, PII/OpSec)

within a FastAPI backend, enabling seamless real-time text analysis and JSON-based communication with a lightweight browser extension frontend.

Implement Non-Intrusive User Guidance: To develop a browser extension that overlays subtle, color-coded icons and informative tooltips (e.g., "This post may contain implicit hate speech") on social media content, providing actionable insights without disrupting the user experience or requiring censorship [11].

Enhance Robustness with Contextual Analysis: To incorporate social context and user data into the hate speech module, leveraging insights from recent studies to improve detection robustness across platforms and languages, addressing generalization challenges [4, 6, 12].

Evaluate System Efficacy and Fairness: To assess the performance of "The Guardian" using metrics like precision, recall, and fairness with a focus on the hate speech module's ability to detect nuanced toxicity while minimizing biases, alongside collecting optional user feedback on usability and trust [2, 14, 15].

6. METHODOLOGY

This methodology outlines the development, implementation, and evaluation of "The Guardian," a modular AI-powered system designed to proactively assist users in identifying human-layer cybersecurity threats on social media in a non-intrusive manner. The approach aligns with the overall project goal of enhancing personal cybersecurity posture while emphasizing the hate speech detection component, which targets explicit and subtle, context-dependent toxicity to improve accuracy and reduce false positives on legitimate critiques [1, 3, 9]. The methodology leverages public datasets, supplemented by custom samples where needed, and employs state-of-the-art technologies to ensure robust, equitable, and user-centric outcomes.

6.1 System Architecture and Data Flow

"The Guardian" is structured as a modular system with a lightweight browser extension frontend and a Python-based FastAPI backend. The architecture integrates four specialized Transformer-based NLP modules (hate speech, PII/OpSec, misinformation, phishing/scams), with the hate speech module as a key focus. The system scrapes text from social media platforms, processes it in real-time, and delivers non-intrusive alerts via color-coded icons and tooltips, ensuring privacy by anonymizing data during transmission.

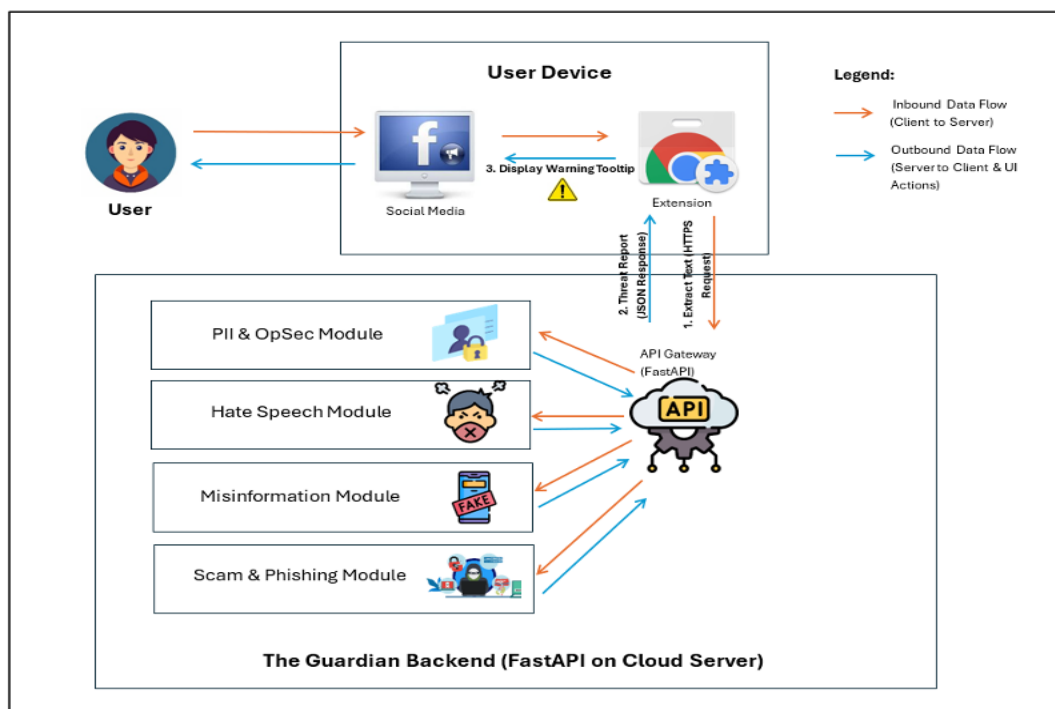


Figure 6.1: System architecture of "The Guardian," showing the data flow from the client-side browser extension to the backend analysis modules and back.

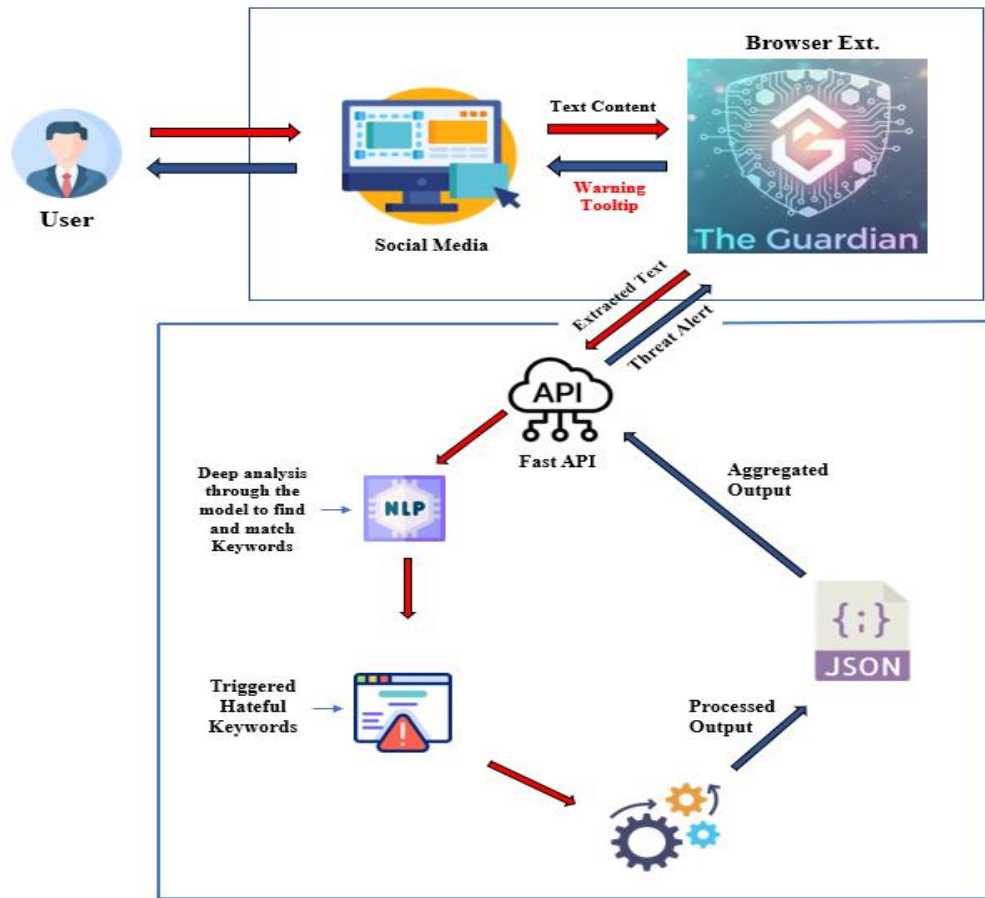


Figure 6.1: Data Processing architecture of Hate Speech module.

6.2 User Flow

- **User Browsing & Text Extraction**

When the user scrolls social media, the browser extension reads the visible text of a post or comment (only the text snippet needed for analysis). With user consent, the extension packages this text and a few context fields (platform, post id) into a small JSON request — it does not store long-term copies on the device unless the user opts in.

- **Request Sent to API Gateway (FastAPI)**

The extension sends the JSON request over HTTPS to the FastAPI gateway. The

gateway authenticates the request, enforces rate limits, and routes the payload to the hate-speech backend for analysis.

- **NLP Model Analysis**

Inside the backend, the core NLP model (a fine-tuned Transformer) processes the incoming text and produces a primary output: a classification label (e.g., Hate / Offensive / Neutral) and a confidence score. This step captures the model's contextual understanding of the message.

- **Keyword Matching / Trigger Detection**

The model output is passed to a keyword matcher that scans the text for known hateful or toxic tokens, coded phrases, and slang variants. The matcher compiles a list of **triggered hateful keywords**—the exact words or phrases that contributed to the decision.

- **Constructing the Trigger List**

The triggered keywords are organized (deduplicated and ranked by importance) to form a concise list that highlights the strongest lexical evidence for the classification. This list helps produce a simple, explainable rationale for the user.

- **JSON Output Formatting**

A JSON Output Formatter packages the final results into one response object. Typical fields include: `label`, `confidence`, `triggered_keywords[]`, and a short `explanation` snippet. This JSON is returned to the API gateway as the aggregated response.

- **Response Delivered Back to the Extension**

The API gateway forwards the JSON back to the browser extension. The extension reads the label and triggered keywords and maps them to a color-coded icon and a brief tooltip message appropriate to the confidence level.

- **User-Facing Tooltip & Interaction**

On the page, the extension shows a subtle icon next to the content; hovering (or clicking) opens a short tooltip that explains why the content was flagged and lists the triggered keywords. The UI is intentionally non-intrusive — it informs and empowers the user rather than blocking content.

6.3 Data Acquisition and Preprocessing

Data Sources: For the hate speech detection module, primary datasets include the Davidson dataset [1], providing labeled Twitter data to distinguish hate speech from offensive and neutral text, and HateXplain [2], offering explainable annotations for toxic spans and rationales. Additional datasets, such as ToxiGen for adversarial implicit hate [10] and Turkish [20], may supplement to address diverse contexts. If gaps exist for detecting nuanced, small custom-labeled samples will be created by annotating real social media posts based on linguistic patterns [3, 9].

Preprocessing: Text data will be cleaned using tools like spaCy, NLTK (Natural Language Toolkit), Scikit-learn for tokenization, lemmatization, stop-word removal, and entity recognition. Special handling for emojis, slang, and coded language will capture implicit toxicity. Data augmentation techniques (e.g., synonym replacement, back-translation) will enhance diversity, particularly for underrepresented subtle forms, drawing on methods to improve model robustness. Preprocessed data will be formatted for Hugging Face Transformers compatibility.

6.4 Software Solution

6.4.1 Requirement Gathering

Overall System: Requirements include real-time text analysis, non-intrusive alerts, privacy preservation, and scalability across threat vectors. The system must integrate with major browsers (e.g., Chrome, Firefox) and platforms (e.g., X, Facebook).

Hate Speech Component: The module requires high accuracy in detecting explicit and implicit toxicity and explain ability for user trust, and fairness to minimize biases across demographics. It must process varied linguistic patterns, including slang and context-dependent phrases, using datasets like HateXplain and Davidson [1, 2].

6.4.2 Feasibility Study

Technical Feasibility: Technologies such as Python, Hugging Face Transformers (BERT, DistilBERT), spaCy, FastAPI, and JavaScript are mature and suitable [5, 12]. Public datasets are accessible, and custom sample creation is feasible with manual annotation [7, 18].

Operational Feasibility: The browser extension ensures seamless integration, and the backend supports scalable processing. The hate speech module’s use of Transformers leverages pre-trained models, reducing development time .

Economic Feasibility: Open-source tools (Hugging Face, spaCy, FastAPI) minimize costs. Cloud hosting for API is viable for prototyping, with local deployment options for efficiency.

6.4.3 Design and Implementation

- **Frontend (Browser Extension):** Developed using JavaScript, HTML, and CSS, the extension scrapes text from posts, comments, and messages in real-time. It sends requests to the backend API and displays color-coded icons with tooltips (e.g., “This post may contain coded hate speech”) based on JSON responses.
- **Backend (FastAPI):** Built with Python and FastAPI, the backend hosts four NLP modules. The hate speech module uses fine-tuned BERT or DistilBERT from Hugging Face, trained on preprocessed datasets to detect explicit and subtle toxicity, incorporating attention mechanisms for toxic spans [2, 5]. Social context integration enhances robustness. Other modules follow similar designs.
- **Hate Speech Module Implementation:**
 - **Model Selection:** Pre-trained BERT/DistilBERT models, fine-tuned for multiclass classification (e.g., hate vs. non-hate, with nuance levels) [1, 12].

- **Training:** Supervised learning with transfer learning on Davidson and HateXplain, augmented for slang and "dog whistles" [3, 10].
 - **Bias Mitigation:** Fairness analysis using metrics like demographic parity, adjusting weights, or oversampling to reduce biases [14, 18].
- **Collaboration:** GitHub manages version control, facilitating code reviews and continuous integration.

6.4.4 Testing

- **Unit Testing:** Test individual components (e.g., text scraping, API endpoints, model predictions) using Postman for API functionality and Jupyter Notebooks for model performance.
- **Integration Testing:** Verify data flow between frontend and backend, ensuring JSON responses and tooltip rendering.
- **Hate Speech Module Testing:** Evaluate on held-out test sets from Davidson and HateXplain, focusing on implicit toxicity detection [1, 2, 9]. Bias testing assesses fairness across subgroups (e.g., gender, race) [14].
- **System Testing:** Validate end-to-end functionality, ensuring real-time analysis across platforms like X [5].

6.5 Practical Constraints and Risk Mitigation

- **Constraints:**
 - **Data Limitations:** Public datasets may lack sufficient examples of subtle "dog whistles" or multilingual slang [3, 9, 16]. **Mitigation:** Create small custom-labeled datasets (e.g., 500 samples) by annotating real social media posts, guided by linguistic patterns [10, 18].
 - **Computational Resources:** Fine-tuning Transformers requires GPU resources [5]. **Mitigation:** Use cloud platforms (e.g., AWS, Google Colab) for training and optimize with DistilBERT for efficiency.
 - **Privacy Concerns:** Users may distrust data transmission to the backend [11]. **Mitigation:** Anonymize text data, use secure HTTPS, and explore on-device processing for future iterations.

- **Bias in Models:** Risk of amplifying demographic biases in hate speech detection [14]. **Mitigation:** Conduct fairness evaluations using demographic parity and equal opportunity metrics, adjusting training data or model weights [14, 18].
- **Multilingual Challenges:** Limited coverage for non-English or code-mixed contexts [16, 20]. **Mitigation:** Incorporate datasets like Hindi-English and Turkish corpora, with potential for multilingual pre-trained models [12].
- **Risk Mitigation:**
 - Regular model retraining to adapt to evolving slang and contexts [3].
 - User feedback loops to refine tooltip clarity and reduce intrusiveness [11].
 - Robust error handling in FastAPI to ensure API uptime and reliability.
 - Backup local processing for API downtime to maintain functionality.

6.6 Evaluation Plan

- **Metrics for Hate Speech Module:**
 - **Precision, Recall, Accuracy:** Measure detection performance on test sets from Davidson and HateXplain, targeting >90% accuracy for explicit hate and >80% recall for implicit toxicity [1, 2, 9].
 - **False Positive/Negative Rates:** Assess errors, particularly false positives on legitimate critiques (<5%) and false negatives on subtle hate (<10%), to ensure user trust [3, 14].
 - **Fairness Metrics:** Evaluate demographic parity and equal opportunity across subgroups (e.g., gender, race), targeting <10% disparity to minimize bias [14, 18].
- **Metrics for Other Modules:** Apply similar metrics (precision, recall, accuracy) to misinformation, phishing/scams, and PII/OpSec modules, ensuring consistent performance across the system.
- **User Interface Feedback:** Collect optional user feedback via surveys within the extension, assessing tooltip clarity, alert intrusiveness, and perceived cybersecurity enhancement [11].

- **Testing Environment:** Use Jupyter Notebooks for model evaluation (e.g., confusion matrices, ROC curves), Postman for API response validation, and real-world simulations on platforms like X to test end-to-end functionality [5].
- **Cross-Dataset Generalization:** Validate model robustness using ToxiGen and multilingual datasets to ensure performance across diverse contexts [4, 10, 16, 20].
- **Iterative Refinement:** Use evaluation results to retrain models, adjust alert thresholds, and refine UI based on user feedback, ensuring alignment with cybersecurity goals of mitigating online harassment [1, 3].

7. PROJECT REQUIREMENTS

The following tables outline the requirements for "The Guardian," a modular AI-powered system designed to provide real-time, non-intrusive guidance against human-layer cybersecurity threats on social media. The system integrates four NLP modules (hate speech, misinformation, phishing/scams, PII/OpSec) within a browser extension frontend and FastAPI backend. The hate speech detection component focuses on identifying explicit and subtle toxicity, including context-dependent "dog whistles," while minimizing false positives on legitimate critiques, ensuring accuracy, fairness, and user empowerment.

7.1 User Requirements

- **Proactive Threat Guidance:** Users shall receive real-time, non-intrusive alerts for detected threats, enabling informed decisions without content blocking. For hate speech, alerts shall highlight explicit slurs and subtle "dog whistles" with educational explanations to foster awareness.
- **Privacy Preservation:** Users shall control data sharing with opt-out options for text analysis. The system shall not require logins or store personal data, ensuring anonymous operation during hate speech detection to protect user privacy.
- **Customizability:** Users shall adjust alert sensitivity (e.g., low, medium, high for hate speech detection) and toggle tooltip visibility to minimize notification overload, tailoring the experience to their preferences.
- **Educational Insights:** Tooltips shall provide concise explanations (e.g., how coded language signals bias in hate speech) to educate users on identified threats, with optional links to cybersecurity tips that do not track user activity.
- **Accessibility:** The interface shall support screen readers, high-contrast modes, and multiple languages for tooltips, ensuring inclusivity for diverse users, including those affected by hate speech.

7.2 Functional Requirements (FR)

Table 7.2: Functional Requirements (FR)

ID	Requirement	Priority
FR1	The browser extension shall scrape text (posts, comments, messages) from social media platforms (e.g., X, Facebook, Instagram) in real-time without altering page layout.	High
FR2	The FastAPI backend shall process scraped text using four NLP modules, returning JSON responses within 2 seconds.	High
FR3	The frontend shall display color-coded icons (e.g., red for high-risk hate speech, orange for subtle toxicity) next to analyzed content.	High
FR4	The backend shall preprocess text using spaCy for tokenization, lemmatization, and entity recognition.	High
FR5	The system shall support independent deployment and updates of each module.	Medium
FR6	The hate speech module shall provide interpretable outputs, highlighting toxic spans and rationales to educate users.	Medium
FR7	The system shall analyze text for all four threats concurrently, prioritizing alerts (e.g., combining hate speech and misinformation).	Medium

7.3 Non-Functional Requirements (NFR)

Table 7.3: Non-Functional Requirements (NFR)

ID	Requirement	Metric
NFR1	Performance: Handle 100 concurrent analyses per session; response time <2 seconds.	Average response time
NFR2	Scalability: Support 1,000 concurrent users with <5% performance degradation	User Response Time Degradation
NFR3	Security and Privacy: 100% of data transmissions use HTTPS; zero persistent storage of user data.	Privacy Leakage
NFR4	Usability: Extension installation time <1 minute; tooltips <50 words.	User feedback
NFR5	Compatibility: Compatibility with Chrome, Firefox, Edge; support for X, Facebook, Instagram; multilingual detection accuracy >75%.	Cross-Platform Success Rate

7.4 Test Cases

Table 7.4: Test Cases

Test Case ID	Test Description	Expected Result
TC-01	Explicit Hate Speech Detection	Backend classifies as “hate speech” with high confidence. Frontend displays a red icon with tooltip: “This post contains explicit hate speech.”
TC-02	Implicit Toxicity Identification ("Dog Whistle")	Backend classifies as “hate speech” with moderate-high confidence. Frontend displays an icon with tooltip: “This post may contain implicit hate speech.”

TC-03	False Positive Avoidance on Legitimate Critiques	Backend classifies as “non-hate”. No icon or tooltip on frontend.
TC-04	Real-Time Integration and User Interface Functionality	Backend processes in <2 seconds.

8. BUDGET AND BUDGET JUSTIFICATION

This section outlines the budget and justification for the development of the hate speech detection component within "The Guardian," a modular AI-powered system for identifying human-layer cybersecurity threats on social media. The budget focuses on the specific requirements for the hate speech module. The budget is structured to leverage open-source tools and public datasets to ensure cost efficiency, aligning with an 8-month timeline starting July 2025.

- **Personnel:** Resources for a researcher or small team to perform tasks specific to the hate speech detection module, including linguistic research, dataset curation, model development, bias analysis, and system integration.
- **Hardware/Cloud Computing:** Access to cloud-based GPU resources for training and fine-tuning Transformer models for the hate speech module.
- **Software/Tools:** Utilization of open-source tools, including Python, Hugging Face Transformers, spaCy, FastAPI, Postman, and Jupyter Notebooks, for development, testing, and evaluation.
- **Testing and Evaluation:** Resources for testing tools (Postman for API testing, Jupyter Notebooks for model evaluation) to validate the hate speech module's performance and integration.

REFERENCES

- [1] Davidson, T., Warmesley, D., Macy, M., & Weber, I. (2017). Automated hate speech detection and the problem of offensive language. In Proceedings of the International AAAI Conference on Web and Social Media (Vol. 11, No. 1). [Link](#)
- [2] Mathew, B., Saha, P., Yimam, S. M., Biemann, C., Goyal, P., & Mukherjee, A. (2021). HateXplain: A benchmark dataset for explainable hate speech detection. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 35, pp. 14867–14875). [Link](#)
- [3] Rawat, A., et al. (2024). Hate speech detection in social media: Techniques, recent trends, and future challenges. WIREs Computational Statistics. [Link](#)
- [4] Fortuna, P., Soler-Company, J., & Wanner, L. (2021). How well do hate speech, toxicity, abusive and offensive language classification models generalize across datasets? In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 576–587). [Link](#)
- [5] Mutanga, R. T., Naicker, N., & Olugbara, O. O. (2020). Hate speech detection in Twitter using transformer methods. International Journal of Advanced Computer Science and Applications, 11(9), 614–620. [Link](#)
- [6] Nagar, S., Barbhuiya, F. A., & Dey, K. (2023). Towards more robust hate speech detection: Using social context and user data. Social Network Analysis and Mining, 13(1), 47. [Link](#)
- [7] Huang, F., Kwak, H., & An, J. (2023). Is ChatGPT better than human annotators? Potential and limitations of ChatGPT in explaining implicit hate speech. In Proceedings of the ACM Web Conference 2023 (pp. 294–297). [Link](#)
- [8] Poletto, F., Basile, V., Sanguinetti, M., Bosco, C., & Patti, V. (2021). Resources and benchmark corpora for hate speech detection: A systematic review. Language Resources and Evaluation, 55(2), 477–523. [Link](#)
- [9] Wiegand, M., Ruppenhofer, J., & Eder, E. (2021). Implicitly abusive language—What does it actually look like and why are we not getting there? In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 576–587). [Link](#)
- [10] Hartvigsen, T., Gabriel, S., Palangi, H., Sap, M., Ray, D., & Kamar, E. (2022). ToxiGen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (pp. 3309–3326). [Link](#)

- [11] Oussalah, M. (2023). A systematic review of hate speech automatic detection using natural language processing. *Neurocomputing*, 546, 126232. [Link](#)
- [12] Aluru, S. S., et al. (2024). A comprehensive review on automatic hate speech detection in the age of the transformer. *Social Network Analysis and Mining*, 14(1), 1–25. [Link](#)
- [13] Das, M., Raj, R., Saha, P., Mathew, B., Gupta, M., & Mukherjee, A. (2023). HateMM: A multi-modal dataset for hate video classification. *arXiv preprint arXiv:2305.03915*. [Link](#)
- [14] Nozza, D., Volpetti, C., & Fersini, E. (2019). Unintended bias in misogyny detection. In *Proceedings of the 2019 IEEE/WIC/ACM International Conference on Web Intelligence* (pp. 149–155). [Link](#)
- [15] Arango, A., Pérez, J., & Poblete, B. (2020). Hate speech detection is not as easy as you may think: A closer look at model validation. *Information Processing & Management*, 57(4), 102226. [Link](#)
- [16] Bohra, A., Vijay, D., Singh, V., Akhtar, S. S., & Shrivastava, M. (2018). A dataset of Hindi-English code-mixed social media text for hate speech detection. In *Proceedings of the Second Workshop on Computational Modeling of People’s Opinions, Personality, and Emotions in Social Media* (pp. 36–41). [Link](#)
- [17] Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N., & Kumar, R. (2019). Predicting the type and target of offensive posts in social media. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 1415–1420). [Link](#)
- [18] Vidgen, B., & Derczynski, L. (2020). Directions in abusive language training data: Garbage in, garbage out. *arXiv preprint arXiv:2004.01670*. [Link](#)
- [19] Sheth, P., et al. (2023). TARGE: Large language model-powered explainable hate speech detection. *arXiv preprint arXiv:2305.16236*. [Link](#)
- [20] Mayda, C., et al. (2021). A multilingual hate speech detection dataset for Turkish social media. In *Proceedings of the 2021 International Conference on Natural Language Processing*. [Link](#)